Mastering the Killing Part: Localized Spark Control in Music Generation

Anonymous submission to Interspeech 2026

Abstract

Language model-based music generation faces a dilemma: play it safe, and the output is highly predictable; turn up the randomness, and the song collapses into chaos. However, the soul of any great track is based on those unpredictable highlights, which we call "sparks". Current methods frequently ruin the entire track when attempting to creating such a moment of innovation. Therefore, we introduce SparkTS, a tuning-free decoding framework based on constrained Monte Carlo Tree Search. Considering general metrics are insufficient for localized innovation evaluation, we also designed three new composite metrics (SCI, SLS, and SCS) for spark-specific analysis. The experiment results show that SparkTS successfully generates intense, perfectly timed musical sparks without sacrificing the overall quality and coherence.

Index Terms: music generation, controllable synthesis, tree search

1. Introduction

Language models depend on next-token prediction for music generation, inherently trapping themselves into a statistical inevitability [1]. Research shows that what actually keeps in our mind are those striking killing parts that can not be predicted [2]. We define these creative musical events as "sparks".

A successful spark should be a surprise under expectations rather than random noise [3–5], which must simultaneously satisfy three requirements: break the normal patterns, rarely occur, and rapidly recover to the stable distribution [6]. Currently, most relevant research try to promote generation diversity simply by uplifting sampling entropy [7,8]. However, this approach is insufficient for local control, as it can hardly schedule when a spark should hit or pull back to safety [9]. As a result, turning up the global randomness, in most cases, ruins the track instead of creating a truly memorable moment [10, 11].

From a control perspective, existing decoding strategies also demonstrate structural limitations [12, 13]. One-step-ahead methods is short-sighted: once they generate a unusual token, they usually fail to guide the audio back to stability again [14]. Contrastive decoding and re-ranking strategies have the potential of spark generation, but they lack necessary control mechanisms [15]. Specifically, localized spark control should be treated as a multi-objective problem: the model must stay in an overall conformity, with a sudden period of creativity happening in an expected span, and then safely land back to the high-probability manifold. Those requirements push the necessity of a multi-step look-ahead structure [16].

Evaluating these sparks is another major challenge [17]. Widely used global metrics are designed to reward overall coherence [18]. But creating a sudden, innovative spark naturally requires the tokens to break this consistency. Consequently, standard metrics fail to recognize creativity. Since these benchmarks are designed to reward predictability, they penalize those rare variations, misidentifying intentional sparks as technical errors [19].

We propose SparkTS, a decoding structure that controls music generation at inference time. Guided by a pre-trained GPT-2 model and a value head, SparkTS produces a brief creative burst and then quickly returns the sequence to a normal flow [20,21]. Considering that existing metrics are not designed for these localized moments, we also introduce a spark-specific evaluation protocol to balance innovation and stability. Overall, our work provides a practical framework for both generating and evaluating musical sparks.

2. Preliminaries and Definitions

A spark is defined as a transient peak in creative deviation that remains within the global acoustic distribution [22]. Let the audio token vocabulary be $\mathcal{V}$ and the training set be $\mathcal{D} = \{x_{1:T_n}^{(n)}\}_{n=1}^{N}$. Given a seed prefix $x_{1:t_0}$, our objective is to generate a melody sequence $\hat{x}_{1:T}$ with a length of T, containing sparks inside [23, 24]. An autoregressive language model generates tokens step-by-step: $\hat{x}_t \sim p_\theta(\cdot \mid \hat{x}_{<t})$ [25]. Standard decoding naturally chooses high-probability tokens, which can hardly bring creative events. To measure how a token deviates from the norm, we define token-level surprisal in Eq. 1 [26]:

$$l_t(\hat{x}) \triangleq -\log p_\theta(\hat{x}_t \mid \hat{x}_{<t}). \tag{1}$$

Global Fidelity and Coherence. A successful spark is required to preserve the overall audio quality and style. Accordingly, we use Fréchet Audio Distance (FAD) [27] to evaluate global fidelity and Acoustic Feature Similarity (SIM) [29] to test stylistic consistency. However, general metrics naturally penalize innovation. A creative spark involves a specific deviation from the standard distribution, so it inherently results in relatively lower scores on FAD and SIM. Consequently, these metrics cannot evaluate innovation individually; they must be paired with spark-specific indicators to distinguish true creativity.

Intense and Sparse Sparks. A spark should be both intense and rare. Let $\mathbf{l}(\hat{x}) = \{l_t(\hat{x})\}_{t=t_0+1}^{T}$ be the sequence of surprisal values. We use the 99th percentile of these values to measure tail intensity in Eq. 2:

$$q_{99}(\hat{x}) \triangleq Q_{0.99}(\mathbf{l}). \tag{2}$$

To measure how often such events occur, we set a threshold τ_{sp} and define a spark indicator $z_t(\hat{x}) = \mathbb{I}[l_t(\hat{x}) > \tau_{\mathrm{sp}}]$. Spark

sparsity is measured as (Eq. 3):

$$\rho(\hat{x}) \triangleq \frac{1}{T - t_0} \sum_{t=t_0+1}^{T} z_t(\hat{x}). \tag{3}$$

To ensure sparks occur as expected, we define allowed innovation spans $\mathcal{S} = \bigcup_{j=1}^{J} [a_j, b_j) \subseteq \{t_0 + 1, \ldots, T\}$. We only expect spark planning inside $\mathcal{S}$, while standard sampling keeps the sequence stable outside it. Timing accuracy is measured by in-span and out-of-span hit rates (Eq. 4):

$$r_{\text{in}}(\hat{x}) = \frac{\sum_{t \in \mathcal{S}} z_t(\hat{x})}{\sum_{t=t_0+1}^{T} z_t(\hat{x}) + \varepsilon}, \qquad r_{\text{out}}(\hat{x}) = \frac{\sum_{t \notin \mathcal{S}} z_t(\hat{x})}{|\{t \notin \mathcal{S}\}|}. \tag{4}$$

High r_{in} and low r_{out} indicate effective span-local control.

Fast Post-Spark Recovery. After a spark, the music should return safely to the normal distribution. For a recovery window of length K following a spark event at t, we calculate the recovery drop in Eq. 5:

$$\Delta(\hat{x}) = \mathbb{E}_{t \in E} \left[l_t - \frac{1}{K} \sum_{i=1}^{K} l_{t+i} \right]. \tag{5}$$

A larger $\Delta(\hat{x})$ means the audio returns to a stable regime more quickly instead of drifting into disorder.

Composite Evaluation Metrics. Each metric above captures only one aspect of music generation, and these objectives often conflict. To summarize the trade-offs more concisely, we introduce three composite scores. First, to penalize models that deviate from a target spark frequency $\rho^\star$, we use a rate-alignment factor $A(\hat{x}) = \frac{\rho^\star}{|\rho(\hat{x}) - \rho^\star| + \rho^\star}$.

Spark Control Index (SCI) measures whether the model can generate intense local variations without degrading the overall audio quality (Eq. 6):

$$\text{SCI}(\hat{x}) = \frac{q_{99}(\hat{x})\, \Delta(\hat{x})}{\text{FAD}(\hat{x})} \cdot A(\hat{x}). \tag{6}$$

Spark Localization Score (SLS) measures whether the model strictly confines sparks to the target span without over-generating them elsewhere (Eq. 7):

$$\text{SLS}(\hat{x}) = \big(r_{in}(\hat{x}) - r_{out}(\hat{x})\big) \cdot A(\hat{x}). \tag{7}$$

Spark Coherence Score (SCS) evaluates whether the intense sparks preserve the original musical style rather than degrading into acoustic noise (Eq. 8):

$$\text{SCS}(\hat{x}) = q_{99}(\hat{x}) \cdot \text{SIM}(\hat{x}) \cdot A(\hat{x}). \tag{8}$$

3. Methodology

3.1. Pipeline Overview

As shown in Fig. 1, the SparkTS pipeline consists of an offline phase for learning acoustic constraints and an online spark inference phase. Designed as a plug-and-play decoding framework, it introduces no new parameters during inference. Outside the target spans, the base language model uses standard sampling to keep the audio progressing smoothly [30]. Once inside a target span, the system switches to Monte Carlo Tree Search (MCTS) [20, 31] to plan the upcoming tokens. Considering audio tokens are highly prone to noise, we guide the tree's leaf evaluation through three main constraints: (1) **structure evaluation** (using n-grams) to avoid unnatural repetitions; (2) **style evaluation** (via a pre-trained value head and KL priors) to ensure overall musical consistency; and (3) a **surprisal gate with explicit rewards** to drive the sudden variation (the spark) and safely return to the norm.

3.2. Learning Acoustic Constraints

To guide the tree search during inference, we first extract intermediate features from a pre-trained MERT model [32] and apply K-means clustering to group the audio into K acoustic clusters [33]. By calculating how often each cluster appears in the training data, we establish a natural baseline distribution $p_{\text{data}} \in \Delta^{K-1}$. We then exponentially tilt p_{data} toward a uniform distribution to create p_{tilt}, giving rarer clusters a fair chance to be selected as targets during inference.

Next, we train a GPT2-based language model (LM) [34] to predict the next acoustic token. We freeze this backbone and then train a lightweight MLP value head $v_\psi(\cdot \mid x_{<t})$ on its final hidden states to predict the acoustic clusters we identified earlier, ensuring macro-level style consistency during inference [21].

3.3. Spark Inference

Dual-Mode Decoding. Our generation strictly follows the predefined time windows $\mathcal{S} = \bigcup_j [a_j, b_j)$. Standard LM sampling is applied outside $\mathcal{S}$. When entering a span, we sample a target acoustic cluster $c^\star \sim p_{\text{tilt}}$. We then start next-token selection like planning a path on a search tree: each node s represents the audio generated so far, and each action a explores appending a different new acoustic token to build the spark.

Tree Selection. To decide which token to explore next, we use the Predictor Upper Confidence bounds applied to Trees (PUCT) algorithm [16]. For a node s, the planner selects the action (Eq. 9):

$$a^* = \arg\max_a \Big(Q(s,a) + c_{\text{puct}}\, P(s,a)\, \frac{\sqrt{N(s)}}{1 + N(s,a)} \Big), \tag{9}$$

$Q(s,a)$ is the experience score, representing how successful this token was during previous search simulations. $P(s,a)$ is the base intuition, representing the model's original top-k safe predictions. The remaining fraction acts as a curiosity bonus for unexplored tokens, encouraging the system to bravely try new paths and avoiding unnatural acoustic noise based on priors.

Tree Expansion & Guardrails. Audio token vocabularies are massive, so we cannot check every possible next token. Thus, we apply progressive widening [35], expanding up to $m(s)$ children, allowing slightly more options if the base language model is highly uncertain. To prevent the audio from breaking or glitching, we introduce a *hard surprisal gate*. If a candidate token is too unnatural with a token-level surprisal $\ell_\theta(a \mid s) = -\log p_\theta(a \mid s)$ exceeding a threshold τ_{hard}, it will be strictly pruned.

Leaf Evaluation & Shaping. Let $s^+ = s \circ a$ be the extended audio sequence after trying the new token a. This new sequence is scored based on three practical rules: structure, style, and the spark itself. First, to maintain structural consistency and avoid repetition, we extract training-set n-grams G_n ($n \in \{2, 3\}$). We apply a penalty $\xi_n(s^+)$ if the token creates a weird acoustic pattern that never appeared in the training set, and a repetition penalty $\zeta(s^+)$ if it just repeats the recent history [11]. Second, for style coherence, the value head scores how well the sequence matches our target style $c^\star$. To ensure the style doesn't

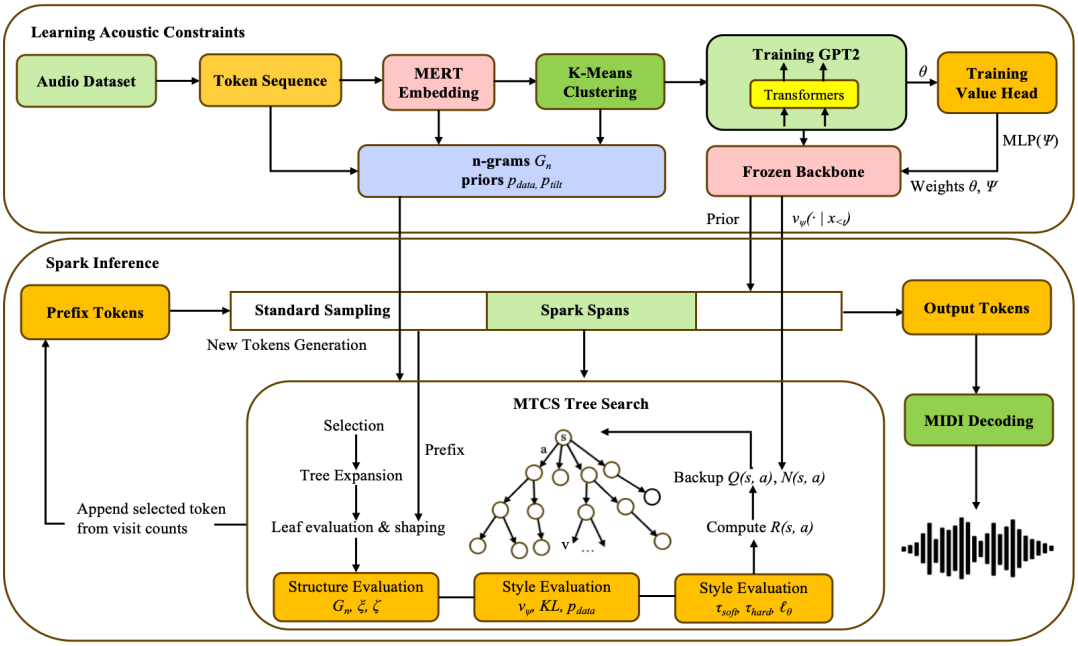

Figure 1: *Overview of the SparkTS pipeline.*

drift too far from reality, we use a KL penalty to restrict the distribution of value head q_ψ close to the original training data (p_{data}). We also add a soft penalty for tokens that are slightly too surprising ($\ell_\theta > \tau_{\text{soft}}$). Finally, we introduce the explicit spark rewards: $R_{\text{spark}}(s, a)$ to encourage a creative peak, and $R_{\text{rec}}(s, a)$ to force a smooth landing back to normal. As shown in Eq. 10, the final score is:

$$\begin{aligned} R(s,a) =& w_v v_\psi(c^\star \mid s^+) + w_2\xi_2(s^+) + w_3\xi_3(s^+) \\ & - w_{\text{rep}}\zeta(s^+) - w_{\text{kl}}[\text{KL}(q_\psi(\cdot \mid s^+)\|p_{\text{data}}) - \text{KL}_{\max}]_+ \\ & - w_{\text{soft}}[\ell_\theta(a \mid s) - \tau_{\text{soft}}]_+ \\ & + R_{\text{spark}}(s,a) + R_{\text{rec}}(s,a). \end{aligned} \tag{10}$$

where $w_{(\cdot)}$ are balancing weights. Our MTCS tree search uses this final score $R(s, a)$ to evaluate paths and update its experience. Once the simulation budget is exhausted, we pick the most visited token as our final choice and keep the relevant part of the tree for the next step.

4. Experiments

4.1. Experiment Setup and Evaluation

We split the POP909 dataset into an 8:1:1 ratio and grouped MERT features into $K = 32$ clusters to represent musical styles [36]. Our backbone is a GPT-2 small model (12 layers, 768 hidden, 512 context), trained with AdamW (LR 2×10^{-4}, effective batch 32) [37]. Then we froze this backbone and trained a separate MLP value head (LR 2×10^{-3}, batch 64) to predict these musical clusters. Then we generated $N = 1000$ sequences, each $T = 800$ tokens long. Every track contains one innovation window ($J = 1$) of 128 tokens, typically spanning from index $t = 300$ to $t = 428$. We aimed for s 3% spark rate ($\rho^\star = 0.03$) and a followed 16-step recovery window ($K = 16$). To keep the audio from quality breaking, we set three safety gates: the 97th percentile for soft warnings (τ_{soft}), the 99th for valid sparks (τ_{sp}), and the 99.9th for hard limit (τ_{hard}).

Inside the innovation spans, we used MCTS with a budget of 192 simulations and a depth/width limit of 64. To stay efficient, we only considered the top-160 tokens (temp 1.05) and applied progressive widening ($k = 2.0$, $\alpha = 0.55$) with a 1.25 entropy coefficient [2]. The final rewards balanced style coherence ($w_{\text{val}} = 1.0$), creativity (novelty $w_2 = 0.02$, $w_3 = 0.15$), and smoothness (repetition $w_{r1} = 0.03$, $w_{r3} = 0.04$). A strict KL-barrier ($w_{\text{KL}} = 0.25$, $\text{KL}_{\max} = 0.35$) ensured the output always sounded like natural music.

For experiment evaluation, we refer to the metrics defined in Section 2, covering base indicators (FAD, SIM, q_{99}, ρ, $r_{\text{in}}/r_{\text{out}}$, Δ), and composite trade-off indices (SCI, SLS, SCS). Additionally, we record the average token-level surprisal $\bar{\ell}_\theta(t)$ for dynamics visualizations. Beyond objective measurements, human aesthetic preference is essential for innovation evaluation. We conducted a blind subjective listening test with 60 participants evaluating 30 audio clips (6 per method) on a 5-point scale [38]. Participants rated three specific perceptual dimensions: MOS-Q (Audio Quality) for the clarity of music; MOS-C (Musical Coherence) for melodic logic and stylistic consistency; and MOS-S (Spark Preference) for the aesthetic impact of the creative sparks.

4.2. Comparative Analysis

We compare SparkTS against four standard decoding methods. For a fair comparison, all methods share the same backbone LM, span schedule, and surprisal thresholds [39]. Outside the spans, every method uses standard autoregressive decoding (temp 1.05, top-$k = 48$) [40]. The only difference is the way they handle the 128-token spark window. Baseline sampling follows what the base model normally predicts [41]. Guided sampling tries to push the generation toward a target style using our value head, but it still only looks one step ahead [42].

Table 1: *Objective and subjective comparison of sparks. ↓ lower is better, ↑ higher is better. Primary metrics are marked with* *.

Method	FAD↓	SIM↑	q_{99} ↑	ρ ↓	Δ ↑	r_{in} ↑	r_{out} ↓	SCI* ↑	SLS* ↑	SCS* ↑	MOS-Q↑	MOS-C↑	MOS-S↑
Baseline Sampling	2.07	0.92	6.65	0.017	6.64	0.18	0.017	15.03	0.11	6.11	3.92	3.85	3.10
Guided Sampling	2.15	0.90	6.84	0.021	6.54	0.16	0.021	16.22	0.11	6.15	4.00	3.95	3.25
Contrastive Decoding	2.75	0.72	9.24	0.108	6.34	0.84	0.021	5.90	0.23	4.20	3.60	3.45	3.90
Greedy Rescoring	3.27	0.55	10.44	0.177	2.55	0.91	0.020	1.38	0.15	2.15	2.85	2.70	3.20
SparkTS (Ours)	2.23	0.88	7.33	0.031	6.64	0.43	0.021	**20.99**	**0.41**	**6.45**	**4.25**	**4.28**	**4.35**

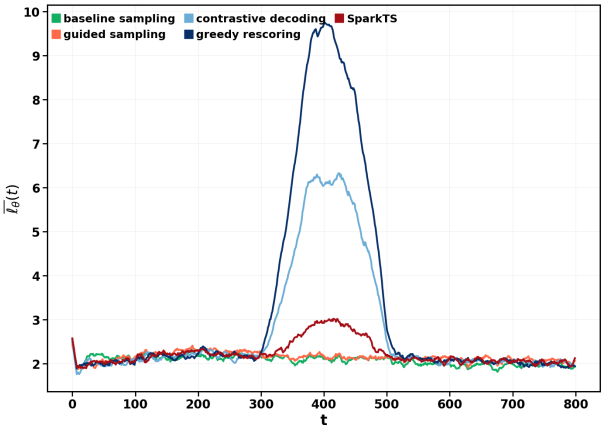

Figure 2: *Average token-level surprisal over time.*

Contrastive decoding forces divergence from typical patterns ($\alpha = 0.5$) [15], while Greedy rescoring aggressively chases the highest spark reward at every single step without looking ahead to see if it destroys the rest of the song.

The objective comparison result reveals a trade-off in audio generation: pushing for extreme creativity often breaks the underlying acoustics, while playing it safe results in boring outputs. On the conservative side, Guided Sampling maintains good audio quality (FAD = 2.15) but is too timid to generate a meaningful highlight: its spark intensity ($q_{99} = 6.84$) and localization (SLS = 0.11) are slightly above the baseline, creating only a marginal overall balance (SCI = 16.22). On the extreme side, while Greedy Rescoring achieves the highest tail intensity ($q_{99} = 10.44$), it fundamentally corrupts the audio. Its FAD spikes to 3.27 (indicating heavy noise/distortion) and its acoustic similarity crashes (SIM = 0.55), causing its innovation-fidelity balance to completely collapse (SCI plummets to 1.38). Contrastive decoding also inflates the spark rate ($\rho = 0.108$), leading to a chaotic listening experience and a severely degraded SCI (5.90). In contrast, SparkTS demonstrates the optimal balance (SCI 20.99). It is the only method that significantly improves spark localization (SLS = 0.41) without sacrificing audio fidelity (FAD = 2.23) or stylistic coherence (SCS = 6.45).

The success also transitions to human perceptual comparisons. In our 60-participant blind test, conservative baselines like Guided Sampling was presented acoustically reasonable (MOS-Q 4.00) but musically dull (MOS-S 3.25). Aggressive methods completely failed: Greedy Rescoring produced unlistenable noise (MOS-Q 2.85, MOS-C 2.70), and Contrastive Decoding just sounded random (MOS-C 3.45). SparkTS, however, easily swept all metrics, covering quality (4.25), coherence (4.28), and innovation (4.35). Listeners didn't hear our sparks as errors. Instead, they treated them as deliberate highlights.

Table 2: *Ablation study of SparkTS components.*

Variant	SCI* ↑	SLS* ↑	SCS* ↑
SparkTS (Full)	**20.99**	**0.41**	**6.45**
w/o novelty shaping	20.45	0.39	6.38
w/o spark shaping	18.31	0.12	6.15
w/o recovery shaping	18.92	0.17	4.12
w/o KL barrier	14.70	0.04	1.85

4.3. Dynamic Control Analysis

Figure 2 demonstrates the average token-level surprisal $\bar{l}_\theta(t)$ over time steps t, with all curves smoothed with a moving average for clarity. A successful spark must act like a controlled explosion. Baseline and guided sampling, respectively produce a relatively flat line. Contrastive decoding and greedy rescoring, on the other hand, completely lose control, showing a dramatic plateau of high surprisal that conspicuously exceeds the target window. SparkTS is the only framework that truly masters the balance of tension and release: it delivers a sharp climax exactly when told, and then immediately guides the music back to a natural flow.

4.4. Ablation Study

We removed key parts of SparkTS one by one to see what would break (Table 2). Removing the **KL barrier** leads to acoustic chaos. Consequently, the spark leaks everywhere, and the overall style coherence completely falls apart (SCS rises to 1.85). Deleting **spark shaping** kills the creativity, as the music falls back to a flat baseline (SCI drops to 18.31). Most crucially, removing **recovery shaping** makes the audio unable to effectively land back onto a normal melody after the climax, leaving the track completely deviate from normal distribution. Finally, removing the acoustic **novelty shaping** (n-grams) only causes a minor drop in overall metrics, but we still choose to keep this part because it acts as a catalyst to generate true innovation.

5. Conclusion

Our research effectively bridges the gap between creative intensity and structural coherence in music generation. SparkTS sweeps all baselines across objective and subjective evaluations, proving that we can deliver striking musical innovations without sacrificing overall quality and coherence. Furthermore, our proposed composite metrics provide a novel paradigm for evaluating localized creativity. Moving forward, since current objective metrics cannot fully capture human taste, future research will focus on aesthetic-driven assessments to harmonize raw creativity with true musical intuition.

6. Generative AI Use Disclosure

Generative AI tools were used exclusively for language editing and proofreading purposes. All research design, analysis, interpretations, and conclusions were independently developed and verified by the author.

7. References

[1] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and Controllable Music Generation," in *NeurIPS*, 2023.

[2] B. P. Gold, M. T. Pearce, E. Mas-Herrero, A. Dagher, and R. J. Zatorre, "Predictability and uncertainty in the pleasure of music: A reward for learning?" *Journal of Neuroscience*, vol. 39, no. 47, pp. 9397–9409, 2019.

[3] D. Huron, *Sweet Anticipation: Music and the Psychology of Expectation*. Cambridge, MA, USA: MIT Press, 2006.

[4] M. T. Pearce and G. A. Wiggins, "Auditory expectation: The information dynamics of music perception and cognition," *Topics in Cognitive Science*, vol. 4, no. 4, pp. 625–652, 2012.

[5] V. K. M. Cheung and S. Sakamoto, "Separating uncertainty from surprise in auditory processing with neurocomputational models: Implications for music perception," *Journal of Neuroscience*, vol. 42, no. 29, pp. 5657–5659, 2022.

[6] S. Abdallah and M. Plumbley, "Information dynamics: Patterns of expectation and surprise in the perception of music," *Connection Science*, vol. 21, no. 2-3, pp. 89–117, 2009.

[7] S. Ji, X. Yang, and J. Luo, "A Survey on Deep Learning for Symbolic Music Generation: Representations, Algorithms, Evaluations, and Challenges," *ACM Computing Surveys*, vol. 56, no. 1, pp. 7:1–7:39, 2024.

[8] F. Carnovalini and A. Rodà, "Computational Creativity and Music Generation Systems: An Introduction to the State of the Art," *Frontiers in Artificial Intelligence*, vol. 3, Art. no. 14, 2020.

[9] C. Meister, T. Pimentel, G. Wiher, and R. Cotterell, "Locally Typical Sampling," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 102–121, 2023.

[10] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The Curious Case of Neural Text Degeneration," in *ICLR*, 2020.

[11] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston, "Neural Text Generation with Unlikelihood Training," in *ICLR*, 2020.

[12] J.-P. Briot and F. Pachet, "Deep learning for music generation: challenges and directions," *Neural Computing and Applications*, vol. 32, no. 4, pp. 981–993, 2020.

[13] D. Herremans, C.-H. Chuan, and E. Chew, "A Functional Taxonomy of Music Generation Systems," *ACM Computing Surveys*, vol. 50, no. 5, pp. 69:1–69:30, 2017.

[14] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck, "Sequence Tutor: Conservative Fine-Tuning of Sequence Generation Models with KL-control," in *ICML*, pp. 1645–1654, 2017.

[15] X. L. Li, A. Holtzman, D. Fried, P. Liang, J. Eisner, T. Hashimoto, L. Zettlemoyer, and M. Lewis, "Contrastive Decoding: Open-ended Text Generation as Optimization," in *ACL*, 2023, pp. 12286–12312.

[16] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. P. Lillicrap, F. Hui, L. Sifre, D. V. Driessche, T. Graepel, and D. Hassabis, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[17] Z. Xiong, W. Wang, J. Yu, Y. Lin, and Z. Wang, "A Comprehensive Survey for Evaluation Methodologies of AI-Generated Music," *CoRR*, vol. abs/2308.13736, 2023.

[18] L.-C. Yang and A. Lerch, "On the evaluation of generative models in music," *Neural Computing and Applications*, vol. 32, no. 9, pp. 4773–4784, 2020.

[19] A. Lerch, C. Arthur, N. Bryan-Kinns, C. Ford, Q. Sun, and A. Vinay, "Survey on the Evaluation of Generative Models in Music," *ACM Computing Surveys*, vol. 58, no. 4, pp. 99:1–99:36, 2026.

[20] L. Kocsis and C. Szepesvári, "Bandit Based Monte-Carlo Planning," in *ECML*, 2006, pp. 282–293.

[21] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. V. Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[22] L. B. Meyer, *Emotion and Meaning in Music*. Chicago, IL, USA: University of Chicago Press, 1956.

[23] A. Roberts, J. H. Engel, C. Raffel, C. Hawthorne, and D. Eck, "A Hierarchical Latent Vector Model for Learning Long-Term Structure in Music," in *ICML*, 2018, pp. 4361–4370.

[24] H.-W. Dong, W.-Y. Hsiao, L.-C. Yang, and Y.-H. Yang, "MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment," in *AAAI*, 2018, pp. 34–41.

[25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *NeurIPS*, 2017, pp. 5998–6008.

[26] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[27] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, "Fréchet Audio Distance: A Reference-Free Metric for Evaluating Music Enhancement Algorithms," in *INTERSPEECH*, 2019, pp. 2350–2354.

[28] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *NeurIPS*, 2017, pp. 6626–6637.

[29] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, "CLAP: Learning Audio Concepts from Natural Language Supervision," in *ICASSP*, 2023, pp. 1–5.

[30] A. Agostinelli, T. I. Denk, Z. Borsos, J. H. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi, M. Sharifi, N. Zeghidour, and C. Frank, "MusicLM: Generating Music From Text," *CoRR*, vol. abs/2301.11325, 2023.

[31] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of Monte Carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

[32] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos, N. Gyenge, R. Dannenberg, R. Liu, W. Chen, G. Xia, Y. Shi, W. Huang, Z. Wang, Y. Guo, and J. Fu, "MERT: Acoustic Music Understanding Model with Large-Scale Self-supervised Training," *CoRR*, vol. abs/2306.00107, 2023.

[33] J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," in *Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability*, 1967, pp. 281–297.

[34] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI Technical Report, 2019.

[35] A. Couëtoux, J.-B. Hoock, N. Sokolovska, O. Teytaud, and N. Bonnard, "Continuous Upper Confidence Trees," in *LION*, 2011, pp. 433–445.

[36] Z. Wang, K. Chen, J. Jiang, Y. Zhang, M. Xu, S. Dai, and G. Xia, "POP909: A Pop-Song Dataset for Music Arrangement Generation," in *ISMIR*, 2020, pp. 38–45.

[37] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.

[38] ITU-R Recommendation BS.1534-3, “Method for the subjective assessment of intermediate quality level of audio systems,” 2015.

[39] Z. Borsos, R. Marinier, D. Vincent, E. Kharitonov, O. Pietquin, M. Sharifi, O. Teboul, D. Grangier, M. Tagliasacchi, and N. Zeghidour, “AudioLM: a Language Modeling Approach to Audio Generation,” *CoRR*, vol. abs/2209.03143, 2022.

[40] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, I. Simon, C. Hawthorne, N. Shazeer, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, “Music Transformer: Generating Music with Long-Term Structure,” in *ICLR*, 2019.

[41] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, “Jukebox: A Generative Model for Music,” *CoRR*, vol. abs/2005.00341, 2020.

[42] G. Hadjeres, F. Pachet, and F. Nielsen, “DeepBach: a Steerable Model for Bach Chorales Generation,” in *ICML*, 2017, pp. 1362–1371.